

Here is the complete catalog of every mistake, hardware state machine conflict, syntax error, and compiler trap identified across your code iterations.

1. I2C Bus Protocol & Address Formatting Errors

- **Unshifted 7-Bit Slave Address:** You loaded the raw address 0x76 directly into the Data Register (DR). The STM32 hardware requires 7-bit slave addresses to be shifted left by 1 bit (address $\ll 1$), reserving Bit 0 for the Read/Write direction. Sending 0x76 unshifted transmitted 0x3B on the bus, causing the sensor to ignore the communication entirely.
- **Missing the Read Phase (Repeated START):** In your first version, you attempted to read data immediately after writing the register address (0xD0), without issuing a Repeated START condition or switching the peripheral to Read Mode (sending the address with Bit 0 set to 1).

2. Flag Check & Hardware Event Sequence Mistakes

- **Checking TxE Instead of SB After START:** Right after setting the START bit, you checked for the Transmit Buffer Empty (TxE) flag. The STM32 hardware state machine requires waiting for the **Start Bit (SB)** flag in Status Register 1 (SR1) before writing to the Data Register.
- **Checking ADDR After Transmitting Data Bytes:** After writing the target register address (0xD0) to the Data Register, you waited for the ADDR flag. The ADDR flag is *only* set by hardware when a slave address match occurs on the bus—it never sets after transmitting standard data bytes. This caused your code to lock up in an infinite loop.
- **Inserting Unnecessary TxE Checks After SB:** In your second iteration, after waiting for the SB flag, you added an extra while loop to wait for TxE before loading the address. In STM32 hardware, the event sequence to clear SB requires reading SR1 during the check and writing directly to DR. Inserting a TxE check between these steps disrupted the hardware state clearance sequence.

3. STM32 State Machine & Timing Pitfalls

- **Incorrect STOP Bit Timing During 1-Byte Read:** You initially cleared the ADDR flag *before* setting the STOP bit in Control Register 1 (CR1). For a single-byte read, STM32 hardware requires you to set the STOP bit (and clear ACK) **before** clearing the ADDR flag. Clearing ADDR first causes the hardware to immediately clock in an unwanted second byte from the sensor, scrambling the transaction.

4. C Syntax & Compiler Optimization Risks

- **Array Assignment Syntax Error:** You initially attempted to assign the Data Register directly to buf (which was declared as `uint8_t buf[8]`). Assigning a single byte value to an array variable name is invalid C syntax. You had to target a specific index, like `buf[0]`.
- **Combining SR1 and SR2 Reads in a Single Bitwise Expression:** You cleared the ADDR flag using `SR1 | SR2` in a single line. In C, combining volatile register reads in one expression allows the compiler to evaluate or reorder them unpredictably. The STM32

hardware strictly requires reading SR1 first, followed by SR2 second.

- **Non-Volatile Dummy Variable Optimization Hazard:** You assigned sequential register reads to a standard global variable (`dummy = SR1; dummy = SR2;`). An optimizing compiler (`-O1`, `-O2`, `-O3`) sees two back-to-back writes to the same variable without an intermediate read, treating the first write as "dead code" and stripping out the SR1 read entirely. This leaves the ADDR flag uncleared and hangs the bus. The variable must be declared with the `volatile` keyword to force the compiler to execute both register reads.

5. Architectural & Hardware Setup Weaknesses

- **Missing GPIO Pull-Up Configuration in Code:** While you set the GPIO pins to Alternate Function Open-Drain, you did not explicitly set the Pull-Up/Pull-Down Register (PUPDR) in software. While your physical board had hardware pull-up resistors, relying on unconfigured internal states makes driver code non-portable to other custom hardware.
- **Unprotected Infinite Blocking Loops:** Every flag wait condition uses a bare `while(!(...))` loop. If a wire disconnects, a sensor crashes, or a NACK occurs, your firmware will freeze permanently in `main()`. A production-ready driver requires timeout counters inside every while loop.